

PROJECT DEFENSE HANDBOOK

Fraud Transaction Detector

Code Map, Business Logic, Database, ML, Security, Operations, and Defense Questions

Guide	Scope
Audience	Student developer, supervisor, external examiner, technical reviewer
Implementation snapshot	Current repository and live SQL schema as reviewed on August 15, 2026
Architecture	Angular UI + Spring Boot modular monolith + Python FastAPI ML service + Microsoft SQL Server
Purpose	Explain where each feature is implemented, why it exists, and how to defend the design honestly

Core claim: The system identifies suspicious transactions and prioritizes investigation. It does not claim that an alert proves fraud; reviewed outcomes provide the labels needed for supervised learning.

Prepared August 15, 2026

How to Use This Handbook

Use the first sections for your presentation and viva answers. Use the middle sections when an examiner asks where a rule is implemented. Use the appendices as a complete lookup for routes, APIs, tables, columns, and configuration keys.

Accuracy of this guide: Current source code and the live SQL schema are the authority. Older phase documents remain useful history, but some mention models and pages that were later replaced.

Contents

- 1. Five-minute defense narrative
- 2. Architecture and repository map
- 3. Authentication, authorization, and security
- 4. Angular pages and user journeys
- 5. Transaction ingestion, feature engineering, and prediction
- 6. Layered risk scoring and decisions
- 7. Unsupervised and supervised learning modes
- 8. Training, comparison, artifacts, and governance
- 9. Labels, cases, false positives, and STR
- 10. Scalability, performance, reliability, and auditability
- 11. Manual demonstration sequence
- 12. Defense questions and honest limitations
- Appendix A. API index
- Appendix B. Complete database table and column reference
- Appendix C. Runtime configuration catalog
- Appendix D. Code navigation cheat sheet

1. Five-Minute Defense Narrative

Problem

Banks process many transactions, confirmed fraud labels are rare and delayed, customer behavior differs, and a single model can create too many false positives. Investigators also need reasons, evidence, reproducibility, and a controlled path from model training to production.

Solution

The project combines point-in-time feature engineering, configurable customer and peer behavior, deterministic AML rules, and a selectable machine-learning ensemble. It supports unsupervised learning when labels are unavailable and supervised learning after reliable labels accumulate. Every suspicious decision can create a case for human review.

Ingest	Build	Score	Decide	Investigate
File or API transaction	Point-in-time features	Behavior + peer + ML + rules	Weighted risk policy	Case, notes, FP, STR

Why the design is defensible

- Spring Boot owns security, SQL data, workflows, audit, and final business decisions; Python owns numerical training and inference.
- Training data is chronological and point-in-time, reducing future leakage.
- Unsupervised comparison uses label-free diagnostics; supervised comparison uses PR-AUC, precision, recall, F1, balanced accuracy, Brier score, and confusion matrices.
- Model output is only one risk layer. Customer behavior, peer behavior, and AML rules remain explicit and explainable.
- Training creates candidates; governance and version pointers separate a trained artifact from a production decision.
- MEDIUM creates a review case; HIGH can generate an STR workflow. Human review remains the final authority.

One-sentence defense: This is an auditable AML decision-support platform that can begin without labels, learn from reviewed outcomes, compare models as data grows, and preserve human control over suspicious-transaction decisions.

2. Architecture and Repository Map

Component	Responsibility	Main location
Angular UI	Forms, reports, comparison views, training operations, risk policy, transaction checks, and cases.	fraud-transaction-ui/src/app
Spring Boot	Authentication, authorization, business workflows, SQL access, features, risk aggregation, alerts, cases, model registry, and audit.	fraud-transaction-detector/src/main/java

Component	Responsibility	Main location
FastAPI ML service	Model training, artifact loading, prediction, growth analysis, agreement, supervised evaluation, and policy backtesting.	fraud-ml-service/app
SQL Server	Authoritative events, labels, features, profiles, configs, model governance, predictions, alerts, and cases.	fraud-transaction-detector/src/main/resources/db

Runtime communication

Browser :4200 JWT + JSON/HTTP	Spring :8080 Business authority	FastAPI :8000 ML computation	SQL Server System of record
---	---	--	---------------------------------------

The browser does not connect directly to SQL Server or FastAPI. This keeps credentials, authorization, audit, and final decision rules in Spring Boot.

Modular monolith decision

The Java services were merged into one Spring Boot application. The code remains separated by packages such as auth, uploads, transactions, aml, comparison, cases, and config. This reduces deployment complexity for the project while preserving module boundaries that can later be separated if scale or organizational ownership requires it.

3. Authentication, Authorization, and Security

Authentication flow

1. A user registers or logs in through Angular.
2. Spring validates the username and BCrypt password hash stored in app_users.
3. Spring signs an HS256 JWT containing subject, role, full name, issued time, and expiry.
4. Angular sends the token as Authorization: Bearer <token>.
5. JwtAuthenticationFilter verifies the signature and expiry, reloads the active user from SQL, and creates the Spring Security principal.
6. SecurityConfig authorizes the route by HTTP method and role. Unauthorized and forbidden responses are structured JSON.

Control	Implementation	Defense point
Password storage	BCrypt with work factor 12	Passwords are not stored in clear text or reversibly encrypted.

Control	Implementation	Defense point
Session model	Stateless JWT	No server HTTP session is required; every protected request is authenticated.
Self-registration	Always creates REVIEWER	A public user cannot grant themselves ADMIN privileges.
API authorization	Spring Security route and method rules	Angular guards improve UX; backend rules provide real enforcement.
Account activation	Active state checked from SQL	A disabled user cannot continue using a previously issued identity unchecked.
CORS	Configured allowed origins and OPTIONS support	Browser cross-origin access is explicit rather than accidental.

Roles

Role	Typical access
REVIEWER	Transaction checks, comparisons, case review, notes, false-positive decisions, STR generation, and manual cases.
ADMIN / AML_ADMIN	Reviewer access plus uploads, training, ML configuration, settings, risk policy, validation, promotion, and rollback.

Key code

```

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/auth/security/SecurityConfig.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/auth/security/JwtAuthenticationFilter.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/auth/service/JwtService.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/auth/service/AuthService.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/auth/service/PasswordHashService.java
fraud-transaction-ui/src/app/core/auth.guard.ts

```

Production hardening: Local database credentials and any development JWT fallback must be replaced by environment variables or a secret manager. Never present a development default as production security.

4. Angular Pages and User Journeys

Route	Purpose	Access	Main component
/login	Simple sign-in page.	Public	login-page.component
/register	Create a REVIEWER account.	Public	register-page.component
/uploads	Upload CSV/Excel transactions; required label guidance changes with system mode.	Authenticated; upload API admin	upload-page.component
/datasets	Unsupervised comparison, agreement, and 10/25/50/100 growth analysis.	Authenticated	datasets-page.component
/supervised-comparison	Supervised model, fusion, confusion-matrix, and full risk-policy performance.	Authenticated	supervised-comparison-page.component
/training-operations	Close dates, create immutable runs, export data, choose configured models, and train candidates.	Admin	training-operations-page.component
/model-tuning	Edit controls for the active supervised or unsupervised model family.	Admin	model-tuning-page.component
/config	Edit four-layer risk weights, thresholds, and production ML composition.	Admin	anomaly-config-page.component
/settings	Select global learning mode and configure cold-start behavior.	Admin	cold-start-config-page.component
/transaction-check	Create/check a single API transaction and inspect explanation, models, and features.	Authenticated	transaction-check-page.component
/cases	Paginated investigation queue, evidence, notes, false positive, and STR actions.	Authenticated	cases-page.component
/cases/manual	Search stored transactions and create a manual investigation case.	Authenticated	manual-case-page.component

Inactive UI scope: Scenario Library, Comparison Runs, and Simulator pages were removed from active Angular routes. Their older backend tables/controllers may remain for compatibility and should not be presented as current user journeys.

5. Transaction Ingestion, Features, and Prediction

Bulk upload

1. Angular sends a CSV or Excel file to Spring.
2. ExcelBulkUploadService normalizes headers, parses rows, validates required fields, records the batch, and writes transactions.

3. Uploaded historical rows preserve FraudLabel when supplied. In supervised mode, null labels are excluded from supervised training.
4. Upload and training remain separate so data can arrive continuously and an administrator can train later.

Single transaction

1. TransactionCreateService rejects a duplicate transaction ID and stores the API event.
2. FeatureEngineeringService loads only information available before the transaction timestamp.
3. The versioned feature vector is persisted in aml_transaction_features.
4. Observed/trusted profile services and learning eligibility are updated according to policy.
5. Cold-start settings may skip ML when trustworthy history is insufficient.
6. AmlPredictionOrchestrator obtains ML evidence and invokes layered scoring.
7. Spring persists fraud_prediction_logs, creates an alert/case when required, and returns risk, action, reasons, model diagnostics, and feature summary.

Feature families

Family	Examples	Reason
Current event	amount, balance, channel, location, type, hour, weekday	Describes the transaction itself.
Customer history	last-5/30 averages, median, std, max/min, debit/credit/cash ratios	Measures deviation from personal behavior.
Velocity	counts and amount sums over 10m, 1h, 24h, 7d, 30d	Detects bursts, rapid movement, and layering.
Novelty	new beneficiary, location, channel, device	Detects first-seen contextual changes.
Peer	peer average, median, std, z-score, frequency percentile	Compares similar customers, including occupation/age-derived groups when available.
Customer profile	risk rating, expected turnover, profile confidence	Controls how strongly behavior evidence should be trusted.
Terminal/context	bounded terminal/location risk representation	Adds operational risk without unbounded one-hot memory growth.

Leakage protection: Historical aggregates must use transaction_date < current transaction date. Training and evaluation use chronological boundaries rather than a random future-aware feature calculation.

Key code

```

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/uploads/service/ExcelBulkUploadService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/transactions/service/TransactionCreateService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/feature/application/FeatureEngineeringService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/feature/infrastructure/FeaturePersistenceService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/profile/application/CustomerProfileService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/learning/application/LearningEligibilityService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/prediction/AmlPredictionOrchestrator.java

```

6. Layered Risk Scoring and Decisions

Layer	Input	Output meaning
Customer behavior	Personal amount, frequency, novelty, time gap, unusual hour, and profile confidence.	How abnormal this event is for the customer.
Peer behavior	Peer amount deviation, frequency percentile, and expected-turnover context.	How abnormal it is for comparable customers.
ML ensemble	Enabled production model scores/decisions and normalized internal weights.	Combined nonlinear evidence from the selected learning mode.
AML rules	Structuring, velocity, amount, turnover, sanctions/location, and other deterministic controls.	Explicit policy evidence; severe rules can override weighted scoring.

Calculation

Each layer produces a normalized score from 0 to 1. The active policy assigns four weights that must total 1.0. The aggregation engine calculates:

Formula: Final risk = customer score × customer weight + peer score × peer weight + ML score × ML weight + rule score × rule weight. A configured hard rule can raise the result to at least the HIGH threshold.

Inside the ML layer, enabled model allocations are normalized to 100%. An anomalous model decision can consume its allocated share, which makes unanimous selected models produce an ML component score of 1.0. The top-level ML weight still limits its contribution to the final score.

Band	Meaning	Typical action
NORMAL	Below LOW threshold	ALLOW
LOW	At or above LOW but below MEDIUM	ALLOW_AND_LOG
MEDIUM	At or above MEDIUM but below HIGH	ALLOW_AND_ALERT; create review case
HIGH	At or above HIGH or hard-rule override	HOLD_FOR_REVIEW; STR workflow may be generated

Why three anomalous ML models may not automatically mean HIGH

Unanimous ML evidence makes the ML component high, not necessarily the final score high. The final score also depends on the ML top-level weight, the other three layers, and the MEDIUM/HIGH thresholds. This is intentional: model agreement is strong evidence, but the business policy controls its operational effect.

Key code

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/prediction/LayeredShadowScoringService
.java
```

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/risk/application/WeightedRiskAggregati
onEngine.java
```

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/behaviour/customer/CustomBehaviourSc
orer.java
```

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/behaviour/peer/PeerBehaviourScorer.jav
a
```

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/rules/engine/DeterministicAmlRuleEngin
e.java
```

```
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/risk/infrastructure/AppConfigRiskPolic
yRepository.java
```

7. Unsupervised and Supervised Learning Modes

The global system.learning_mode setting controls upload guidance, model-tuning controls, training selections, production ML catalog, and the appropriate comparison page.

Mode	When to use	Current models	Valid evaluation claim
UNSUPERVISED	No reliable FraudLabel exists.	Isolation Forest, Autoencoder, Behavioral Cluster Outlier	Label-free separation, anomaly-rate control, agreement, stability, growth, latency; not fraud accuracy.
SUPERVISED	Reliable 0/1 labels exist from source data or reviewed outcomes.	XGBoost, Class-Balanced Random Forest, Extra Trees; optional Temporal Stacked Ensemble	PR-AUC, precision, recall, F1, balanced accuracy, Brier score, confusion matrices, and risk-policy backtest.

Why two modes are necessary

- A new client may have transactions but no confirmed fraud labels, so unsupervised learning provides an honest starting point.
- After investigators mark false positives or suspicious/STR outcomes, reviewed evidence can create supervised labels.
- Rows with FraudLabel = NULL are excluded from supervised training; they are not silently assumed legitimate.
- A new API transaction that creates no case may become label 0 under the accepted business rule; uploaded unlabeled history remains NULL unless the file supplies a label.
- The modes share feature governance and risk layers, but their ML training and evaluation logic differ.

Avoid confirmation bias: Do not convert every model alert into label 1. The label must come from reviewed evidence or an accepted business outcome, otherwise the supervised model only learns the previous model's mistakes.

8. Training, Comparison, Artifacts, and Governance

Training Operations

1. Close business dates so the historical window cannot change during export.
2. Create a training run with feature version, date range, cutoff, and selected models from the active learning mode.
3. Export eligible rows in keyset-paginated chunks to immutable Parquet parts with checksums.
4. Train each selected model independently on the same snapshot.
5. Write immutable artifact bundles, metrics, thresholds, feature columns/schema, and checksums.
6. Register candidate versions. A candidate is not production simply because training completed.

Supervised split

The supervised pipeline uses chronological data: approximately 60% fit, 10% tuning, 10% threshold calibration, and 20% untouched evaluation. This keeps threshold selection and final reporting separate from model fitting.

Unsupervised growth comparison

The oldest 10%, 25%, 50%, and 100% of history are compared. Partitions under the minimum row requirement are skipped. The report shows how detector behavior changes as data grows, including separation/EM-AUC-style diagnostics, anomaly rate, score distribution, training time, throughput, agreement, and stability.

Supervised growth comparison

The same chronological growth idea is applied to labeled rows. Individual models, fusion strategies, confusion matrices, and the complete four-layer risk policy are evaluated on held-out rows. Results are cached in `aml_supervised_growth_studies` so page reloads do not recompute a completed study.

Model tuning versus risk policy

Control	What it changes	What it does not change
Model Tuning	Training hyperparameters, evaluation protocol, enabled candidate generation.	It does not automatically select a model for production.
Risk Policy	Four layer weights, internal ML allocations, LOW/MEDIUM/HIGH thresholds, rule thresholds.	It does not retrain model parameters.
Training Operations	Which configured models are trained against a	It does not activate a candidate.

Control	What it changes	What it does not change
	selected immutable window.	
Model registry/deployment	Which validated immutable model version is active and rollback history.	It does not rewrite historical metrics or artifacts.

Large-data protections

- SQL keyset pagination and Parquet record batches avoid loading all eligible rows in Java memory.
- High-cardinality terminal/location data is compacted into bounded deterministic buckets instead of producing more than ten thousand object columns.
- Tuning uses a bounded representative sample and bounded tree counts; the final selected fit can use the approved full snapshot.
- Comparison result caching prevents repeated expensive recomputation after page refresh.
- Training and comparisons are asynchronous; status is persisted and polled by the UI.

Key code

```

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/training/application/TrainingPipelineJ
obLauncher.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/training/application/TrainingDatasetEx
portService.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/training/application/ProductionCandida
teTrainingService.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/training/application/AmlModelRegistryS
ervice.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/research/application/GrowthAnalysisSer
vice.java
fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/aml/research/application/SupervisedGrowthS
tudyService.java
fraud-ml-service/app/training.py
fraud-ml-service/app/supervised_training.py
fraud-ml-service/app/research/growth_analysis.py
fraud-ml-service/app/research/supervised_growth_analysis.py
fraud-ml-service/app/research/risk_policy_backtest.py
fraud-ml-service/app/feature_compaction.py

```

9. Labels, Cases, False Positives, and STR

Label lifecycle

Situation	FraudLabel	Training effect
Historical upload without label	NULL	Excluded from supervised training; still usable for unsupervised learning.
Historical upload with source label	0 or 1	Eligible if it passes all other learning controls.
New API transaction with no case	0 under the accepted business rule	Can become legitimate supervised evidence after persistence and eligibility checks.
Case open/unreviewed	Pending or unchanged	Do not treat a generated alert as confirmed fraud.
Marked false positive	0	Adds reviewed legitimate evidence and updates linked alert/case state.
Suspicious outcome / STR generated	1 under the accepted project rule	Adds positive evidence; production institutions should align this with formal investigation policy.

Automatic case flow

1. A MEDIUM or HIGH final production risk creates or reuses a case linked to the transaction and alert.
2. The case stores status, priority, assignment, creation source, and the latest decision evidence.
3. The analyst reads user-friendly reasons, component scores, model diagnostics, feature summary, and policy/model versions.
4. The analyst adds notes and either marks false positive or generates STR XML; mutually invalid transitions are blocked.

Case performance controls

- Paginated queries bound the number of cases returned.
- Search and status filters are applied by the API instead of requiring a manual refresh-only client workaround.
- Summary/evidence loading avoids per-row N+1 queries in the queue.
- Duplicate manual or automatic cases for the same event are handled idempotently where applicable.

Key code

```

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/cases/service/CaseManagementService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/cases/service/StrXmlService.java

fraud-transaction-
detector/src/main/java/com/ftd/fraud_transaction_detector/transactions/service/TransactionFraudLabel
Service.java

fraud-transaction-ui/src/app/pages/cases-page.component.ts

fraud-transaction-ui/src/app/pages/manual-case-page.component.ts

```

10. Scalability, Performance, Reliability, and Auditability

Problem	Implemented response	Remaining production work
Millions of rows	Chunked SQL export, Parquet, bounded feature compaction, asynchronous jobs, cached comparisons.	Distributed object storage/compute, workload scheduling, and production capacity testing.
Slow single transaction	Persisted profiles/recent history, bounded lookups, in-memory Python artifacts, and no retraining on request.	Profile cache, connection-pool tuning, p95/p99 tracing, model serving replicas.
ML unavailable	Orchestrator/logging surfaces failure and preserves business control; current path has a fail-safe response where configured.	Circuit breaker, retries with budgets, queueing, and explicit operational SLOs.
Schema drift	Feature version, model feature schema, stored feature JSON, artifact metadata, and registry validation.	Automated migration pipeline and compatibility contracts in CI/CD.
Model risk	Candidate registry, validation, exact-version pointers, promotion/rollback, shadow studies, immutable metrics.	Independent model-risk approval and periodic monitoring governance.
Explainability	Reason codes, component scores, model payloads, feature summary, policy and model version in logs/cases.	SHAP or model-specific local explanations for supervised decisions.
Audit	Transactions, features, prediction logs, labels, cases, notes, STR, validations, deployments, and configs are persisted.	Retention, encryption, tamper-evidence, SIEM integration, and access reviews.

Kafka position: Kafka is not required to prove the current MVP. At bank scale it can decouple ingestion, feature updates, scoring, alerts, and retraining triggers; support replay; and absorb bursts. SQL remains the system of record and consumers must be idempotent.

11. Manual Demonstration Sequence

1. Open /login and authenticate. Explain JWT, BCrypt, roles, and backend enforcement.
2. Open /settings. Choose UNSUPERVISED or SUPERVISED and explain why the whole application follows this mode.

3. Open /uploads. Upload a matching dataset and explain label rules.
4. Open /training-operations. Close required dates, create a run, export the immutable snapshot, choose models, and train candidates.
5. Open the matching comparison page. Show 10%, 25%, 50%, and 100% growth; explain only metrics valid for that mode.
6. Open /model-tuning. Explain that these controls affect the next training/comparison, not current production artifacts.
7. Open /config. Explain four-layer weights, internal ML allocation, thresholds, and why totals must be exactly 100%. Save creates a versioned policy.
8. Open /transaction-check. Submit a realistic event and explain the final score, reasons, component scores, model diagnostics, feature contract, and latency.
9. Open /cases. Show pagination, evidence, notes, false-positive action, and STR XML workflow.
10. Finish with model registry, audit tables, limitations, and how reviewed outcomes enable supervised learning.

Demo advice: Do not start a long comparison during the defense. Run it beforehand and demonstrate the cached completed result. Keep one small dataset available for a short live workflow.

12. Defense Questions and Honest Answers

Common examiner questions

Why not call every anomaly fraud?

An anomaly is only unusual. Fraud requires reliable labels or human investigation. The platform creates suspicious cases rather than claiming proof.

Why both supervised and unsupervised learning?

Some clients begin without labels. Unsupervised models provide a safe starting point; reviewed outcomes later support supervised classifiers and valid precision/recall metrics.

Why PR-AUC instead of only accuracy?

Fraud is imbalanced. A model can achieve high accuracy by predicting legitimate almost always. PR-AUC focuses on positive-class ranking quality.

Why chronological splits?

Random splits can leak future behavioral patterns. Chronology better approximates training on the past and evaluating on later transactions.

Why can the ensemble be worse than one model?

Ensembling only helps when models contribute complementary information and weights/thresholds are validated. Correlated errors or poor calibration can reduce performance.

Does a 90% target manipulate results?

A target is acceptable only as a business operating point chosen on calibration data and honestly reported with its recall and case volume. It must not be tuned on the final test set.

Why keep rules if ML exists?

Rules encode known AML obligations and provide deterministic, explainable overrides. ML discovers nonlinear patterns but should not replace explicit policy.

How does data growth analysis help?

It shows whether quality, rate, agreement, latency, and stability improve or deteriorate as the chronological history increases from 10% to 100%.

How is reproducibility achieved?

Training windows, cutoff, feature version, Parquet checksums, artifact checksums, random seed, metrics, policies, and model versions are persisted.

Why is Java the business authority?

Security, SQL consistency, labels, cases, thresholds, and regulatory workflow belong in a transactional application layer. Python remains focused on ML computation.

Can this handle a bank's full volume today?

The design contains bank-scale patterns, but production capacity is not proven until load tests, distributed deployment, observability, resilience, security hardening, and regulatory validation are completed.

What is the biggest academic contribution?

A dual-learning, growth-aware, layered risk architecture that compares models honestly, preserves explainable AML logic, and connects investigation outcomes back to future supervised learning.

Known limitations

- Synthetic or limited project datasets cannot prove performance on a real bank population.
- STR XML is a draft artifact and not a certified regulator submission integration.
- Production secrets, encryption, retention, privacy, and infrastructure controls require institutional deployment work.
- A high score is decision support; model drift, reviewer consistency, and label quality must be monitored continuously.
- Some compatibility controllers/tables remain although their Angular pages are inactive; they should be cleaned after migration assurance.

Appendix A. API Index

Area	Endpoint	Purpose	Access
Auth	POST /api/auth/register	Create REVIEWER account	Public
Auth	POST /api/auth/login	Issue JWT	Public
Auth	GET /api/auth/me; GET /validate	Current identity/token check	Authenticated
Upload	POST /api/v1/uploads/transactions	Import CSV/Excel	Admin
Upload	GET /api/v1/uploads/batches[/latest/{batchNo}]	Upload history	Authenticated
Transactions	POST /api/v1/transactions	Persist, feature, score, alert/case	Authenticated
Transactions	GET /api/v1/transactions[/{transactionId}]	Search/read transactions	Authenticated
Cases	GET /api/cases	Paginated/filter case queue	Authenticated
Cases	POST /api/cases; GET /api/cases/{id}	Manual create/read detail	Authenticated
Cases	PUT /api/cases/{id}/status; POST /notes	Workflow and notes	Authenticated
Cases	POST /api/cases/{id}/false-positive	Resolve as false positive	Authenticated
Cases	POST /api/cases/{id}/str-xml	Generate draft STR XML	Authenticated
Training	POST /api/v1/aml/training-runs	Create governed run	Admin
Training	POST /api/v1/aml/training-runs/pipeline/start	Create/export/train pipeline	Admin
Training	POST .../{runId}/dataset; POST .../training	Export/train selected run	Admin
Training	GET /api/v1/aml/training-runs	Run status/history	Authenticated
Business day	POST /api/v1/aml/business-days/close[/range]	Seal dates	Admin
Registry	GET /api/v1/aml/models; GET .../active	Registry and active pointers	Authenticated
Registry	POST .../{version}/validate promote rollback	Governed lifecycle	Admin
Research	POST /api/v1/aml/growth-analysis/training-runs/{id}	Unsupervised growth	Admin
Research	POST /api/v1/aml/growth-analysis/layer-ablation	Layer effect replay	Admin
Research	GET/POST /api/v1/aml/growth-studies	Cached unsupervised studies	Auth/Admin
Research	GET/POST /api/v1/aml/supervised-growth-studies	Cached supervised studies	Auth/Admin
Research	GET/POST /api/v1/aml/agreement-studies	Agreement studies	Auth/Admin
Layered	POST /api/v1/aml/layered-shadow/validate	Validate full architecture	Admin
Layered	POST /api/v1/aml/layered-deployments/promote rollback	Layered routing control	Admin
Config	/api/v1/anomaly-model-comparisons/risk-policy	Risk weights/thresholds	Auth/Admin writes
Config	/api/v1/anomaly-model-comparisons/model-tuning	Tuning controls	Auth/Admin writes
Config	/api/v1/anomaly-model-comparisons/model-catalog	Mode-specific model catalog	Authenticated
Python	POST /api/v2/fraud/predict	Production ML inference	Internal Spring

Area	Endpoint	Purpose	Access
			call
Python	POST /api/v1/aml/research/growth-analysis	Unsupervised research compute	Internal Spring call
Python	POST /api/v1/aml/model-agreement	Agreement compute	Internal Spring call

Appendix B. Complete Database Table and Column Reference

Live inventory: 38 tables. Data types, nullability, and identity flags were read from the active fraud-transaction-detector SQL Server schema. Table descriptions summarize current application use.

Identity, configuration, and ingestion

app_users

Application identities, BCrypt password hashes, role, and activation state.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	username	varchar(100)	No	No
3	password_hash	varchar(255)	No	No
4	full_name	varchar(150)	No	No
5	role_name	varchar(50)	No	No
6	is_active	bit	No	No
7	created_at	datetime2	No	No

app_config

Typed runtime configuration used by settings, tuning, research, and risk policy services.

#	Column	SQL type	Nullable	Identity
1	config_key	varchar(100)	No	No
2	config_value	nvarchar(MAX)	No	No
3	value_type	varchar(30)	No	No
4	description	varchar(500)	Yes	No
5	updated_at	datetime2	No	No

bulk_upload_batches

One audit record per uploaded file, including success and failure counts.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	batch_no	varchar(50)	No	No
3	file_name	varchar(255)	Yes	No
4	total_rows	int	No	No
5	success_rows	int	No	No
6	failed_rows	int	No	No
7	uploaded_by	varchar(100)	Yes	No
8	uploaded_at	datetime2	No	No
9	status	varchar(30)	No	No

uploaded_datasets

Logical upload or database snapshot metadata used by comparison workflows.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	dataset_no	varchar(50)	No	No
3	file_name	varchar(255)	No	No
4	total_rows	int	No	No
5	uploaded_by	varchar(100)	Yes	No
6	uploaded_at	datetime2	No	No
7	status	varchar(30)	No	No
8	notes	varchar(500)	Yes	No
9	source_batch_id	bigint	Yes	No
10	source_type	varchar(30)	No	No
11	snapshot_max_transaction_id	bigint	Yes	No

transactions

Authoritative transaction events, source metadata, processing state, and supervised label lifecycle.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	transaction_id	varchar(50)	No	No
3	account_id	varchar(50)	No	No
4	transaction_amount	decimal(18,2)	No	No
5	transaction_type	varchar(50)	No	No
6	transaction_date	datetime2	No	No
7	location	varchar(100)	No	No
8	channel	varchar(50)	No	No
9	customer_age	int	Yes	No
10	customer_occupation	varchar(100)	Yes	No
11	login_attempts	int	No	No
12	account_balance	decimal(18,2)	No	No
13	source_type	varchar(30)	No	No
14	upload_batch_id	bigint	Yes	No
15	created_at	datetime2	No	No
16	customer_id	varchar(100)	Yes	No
17	business_date	date	Yes	No
18	processing_status	varchar(30)	No	No

transactions (continued)

#	Column	SQL type	Nullable	Identity
19	feature_status	varchar(30)	No	No
20	prediction_status	varchar(30)	No	No
21	updated_at	datetime2	No	No
22	fraud_label	bit	Yes	No
23	label_source	varchar(50)	Yes	No
24	labeled_by	varchar(100)	Yes	No
25	labeled_at	datetime2	Yes	No

Features, profiles, and learning

aml_transaction_features

Persisted point-in-time feature vectors used for reproducible scoring and training.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	transaction_id	varchar(50)	No	No
3	customer_id	varchar(100)	No	No
4	account_id	varchar(50)	No	No
5	business_date	date	No	No
6	transaction_date	datetime2	No	No
7	feature_version	varchar(30)	No	No
8	current_amount	decimal(19,4)	No	No
9	current_balance	decimal(19,4)	Yes	No
10	amount_balance_ratio	float	Yes	No
11	transaction_hour	int	No	No
12	transaction_day_of_week	int	No	No
13	is_night	bit	No	No
14	is_weekend	bit	No	No
15	customer_history_count	bigint	No	No
16	trusted_history_count	bigint	No	No
17	recent_transaction_count	int	No	No
18	profile_confidence	float	No	No

aml_transaction_features (continued)

#	Column	SQL type	Nullable	Identity
19	last_5_avg_amount	float	Yes	No
20	last_5_median_amount	float	Yes	No
21	last_30_avg_amount	float	Yes	No
22	last_30_median_amount	float	Yes	No
23	last_30_std_amount	float	Yes	No
24	last_30_max_amount	float	Yes	No
25	last_30_min_amount	float	Yes	No
26	amount_vs_last_30_avg	float	Yes	No
27	amount_vs_last_30_median	float	Yes	No
28	amount_z_score_last_30	float	Yes	No
29	last_30_debit_ratio	float	Yes	No
30	last_30_credit_ratio	float	Yes	No
31	last_30_cash_ratio	float	Yes	No
32	last_30_unique_beneficiaries	int	Yes	No
33	last_30_unique_locations	int	Yes	No
34	last_30_unique_channels	int	Yes	No
35	last_30_avg_time_gap_minutes	float	Yes	No
36	transaction_count_10m	int	No	No

aml_transaction_features (continued)

#	Column	SQL type	Nullable	Identity
37	transaction_count_1h	int	No	No
38	transaction_count_24h	int	No	No
39	transaction_count_7d	int	No	No
40	transaction_count_30d	int	No	No
41	amount_sum_10m	float	No	No
42	amount_sum_1h	float	No	No
43	amount_sum_24h	float	No	No
44	amount_sum_7d	float	No	No
45	amount_sum_30d	float	No	No
46	unique_beneficiaries_1h	int	No	No
47	unique_beneficiaries_24h	int	No	No
48	unique_beneficiaries_7d	int	No	No
49	repeated_amount_count_24h	int	No	No
50	below_threshold_count_24h	int	No	No
51	below_threshold_amount_sum_24h	float	No	No
52	new_beneficiary	bit	No	No
53	new_location	bit	No	No
54	new_channel	bit	No	No

aml_transaction_features (continued)

#	Column	SQL type	Nullable	Identity
55	new_device	bit	No	No
56	unusual_transaction_hour	bit	No	No
57	time_since_previous_transaction_minutes	float	Yes	No
58	peer_group_code	varchar(50)	Yes	No
59	peer_avg_amount	float	Yes	No
60	peer_median_amount	float	Yes	No
61	peer_std_amount	float	Yes	No
62	amount_vs_peer_avg	float	Yes	No
63	peer_amount_z_score	float	Yes	No
64	peer_frequency_percentile	float	Yes	No
65	customer_type	varchar(50)	Yes	No
66	customer_risk_rating	varchar(30)	Yes	No
67	expected_monthly_turnover	float	Yes	No
68	amount_vs_expected_turnover	float	Yes	No
69	generated_at	datetime2	No	No
70	generator_service_version	varchar(50)	No	No
71	model_feature_schema	varchar(50)	No	No
72	model_features_json	nvarchar(MAX)	No	No

aml_transaction_features (continued)

#	Column	SQL type	Nullable	Identity
73	terminal_tx_count_1d	bigint	No	No
74	terminal_fraud_count_1d	bigint	No	No
75	terminal_fraud_rate_1d	float	No	No
76	terminal_avg_amount_1d	float	No	No
77	terminal_tx_count_7d	bigint	No	No
78	terminal_fraud_count_7d	bigint	No	No
79	terminal_fraud_rate_7d	float	No	No
80	terminal_avg_amount_7d	float	No	No
81	terminal_tx_count_30d	bigint	No	No
82	terminal_fraud_count_30d	bigint	No	No
83	terminal_fraud_rate_30d	float	No	No
84	terminal_avg_amount_30d	float	No	No
85	terminal_risk_available	bit	No	No

aml_feature_learning_status

Eligibility, quarantine, review, and learning-state decisions per transaction.

#	Column	SQL type	Nullable	Identity
1	transaction_id	varchar(50)	No	No
2	eligibility_status	varchar(30)	No	No
3	eligibility_reason	varchar(500)	Yes	No
4	eligible_for_incremental_model	bit	No	No
5	eligible_for_trusted_profile	bit	No	No
6	eligible_for_batch_training	bit	No	No
7	reviewed_by	varchar(100)	Yes	No
8	reviewed_at	datetime2	Yes	No
9	updated_at	datetime2	No	No

aml_customer_observed_profile

Fast operational profile updated from observed customer activity.

#	Column	SQL type	Nullable	Identity
1	customer_id	varchar(100)	No	No
2	total_transaction_count	bigint	No	No
3	total_transaction_amount	decimal(19,4)	No	No
4	last_transaction_id	varchar(50)	Yes	No
5	last_transaction_date	datetime2	Yes	No
6	last_transaction_amount	decimal(19,4)	Yes	No
7	last_location_code	varchar(100)	Yes	No
8	last_channel	varchar(50)	Yes	No
9	last_beneficiary_id	varchar(100)	Yes	No
10	observed_avg_amount	float	Yes	No
11	observed_std_amount	float	Yes	No
12	observed_max_amount	float	Yes	No
13	updated_at	datetime2	No	No
14	row_version	timestamp	No	No

aml_customer_trusted_profile

Profile derived only from transactions approved for trusted learning.

#	Column	SQL type	Nullable	Identity
1	customer_id	varchar(100)	No	No
2	trusted_transaction_count	bigint	No	No
3	trusted_avg_amount	float	Yes	No
4	trusted_variance_amount	float	Yes	No
5	trusted_std_amount	float	Yes	No
6	trusted_max_amount	float	Yes	No
7	trusted_min_amount	float	Yes	No
8	usual_start_hour	int	Yes	No
9	usual_end_hour	int	Yes	No
10	dominant_channel	varchar(50)	Yes	No
11	dominant_location_code	varchar(100)	Yes	No
12	profile_confidence	float	No	No
13	profile_status	varchar(30)	No	No
14	last_learned_transaction_id	varchar(50)	Yes	No
15	last_learned_at	datetime2	Yes	No
16	updated_at	datetime2	No	No
17	row_version	timestamp	No	No

aml_customer_recent_transactions

Bounded recent-history support for velocity and behavior calculations.

#	Column	SQL type	Nullable	Identity
1	customer_id	varchar(100)	No	No
2	transaction_id	varchar(50)	No	No
3	transaction_date	datetime2	No	No
4	transaction_amount	decimal(19,4)	No	No
5	transaction_type	varchar(50)	Yes	No
6	channel	varchar(50)	Yes	No
7	location_code	varchar(100)	Yes	No
8	beneficiary_id	varchar(100)	Yes	No
9	device_id	varchar(200)	Yes	No
10	trusted_flag	bit	No	No
11	anomaly_risk_level	varchar(20)	Yes	No
12	inserted_at	datetime2	No	No

aml_business_days

Open/closed business dates used to prevent mutable historical exports.

#	Column	SQL type	Nullable	Identity
1	business_date	date	No	No
2	status	varchar(20)	No	No
3	closed_at	datetime2	Yes	No
4	closed_by	varchar(100)	Yes	No
5	updated_at	datetime2	No	No

Predictions, alerts, cases, and reports

fraud_prediction_logs

Immutable request/response and explanation audit for each scored transaction.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	transaction_id	varchar(50)	No	No
3	account_id	varchar(50)	No	No
4	risk_level	varchar(20)	Yes	No
5	anomaly_votes	int	Yes	No
6	request_json	nvarchar(MAX)	Yes	No
7	response_json	nvarchar(MAX)	Yes	No
8	created_at	datetime2	No	No
9	feature_version	varchar(30)	Yes	No
10	model_version	varchar(100)	Yes	No
11	customer_statistical_score	float	Yes	No
12	incremental_model_score	float	Yes	No
13	batch_model_score	float	Yes	No
14	rule_score	float	Yes	No
15	final_risk_score	float	Yes	No
16	suspicious_flag	bit	Yes	No
17	learning_decision	varchar(30)	Yes	No
18	learning_decision_reason	varchar(500)	Yes	No

fraud_prediction_logs (continued)

#	Column	SQL type	Nullable	Identity
19	reason_codes	nvarchar(MAX)	Yes	No
20	prediction_started_at	datetime2	Yes	No
21	prediction_completed_at	datetime2	Yes	No
22	prediction_duration_ms	bigint	Yes	No
23	risk_policy_version	varchar(50)	Yes	No

fraud_alerts

Suspicious-event alerts created from qualifying production decisions.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	alert_no	varchar(50)	No	No
3	transaction_id	varchar(50)	No	No
4	account_id	varchar(50)	No	No
5	risk_level	varchar(20)	No	No
6	anomaly_votes	int	No	No
7	iso_anomaly	bit	No	No
8	lof_anomaly	bit	No	No
9	svm_anomaly	bit	No	No
10	anomaly_reason	nvarchar(MAX)	Yes	No
11	recommended_action	varchar(50)	Yes	No
12	review_status	varchar(30)	No	No
13	reviewed_by	varchar(100)	Yes	No
14	reviewed_at	datetime2	Yes	No
15	str_file_path	varchar(500)	Yes	No
16	str_generated_at	datetime2	Yes	No
17	created_at	datetime2	No	No

case_records

Investigation case header, workflow state, assignment, priority, and linked transaction/alert.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	case_no	varchar(50)	No	No
3	fraud_alert_id	bigint	Yes	No
4	transaction_id	varchar(50)	No	No
5	account_id	varchar(50)	No	No
6	title	varchar(200)	No	No
7	status	varchar(30)	No	No
8	priority	varchar(20)	No	No
9	assigned_to	varchar(100)	Yes	No
10	created_by	varchar(100)	Yes	No
11	created_at	datetime2	No	No
12	updated_at	datetime2	Yes	No

case_notes

Attributed analyst and system notes attached to a case.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	case_record_id	bigint	No	No
3	note_text	nvarchar(MAX)	No	No
4	created_by	varchar(100)	No	No
5	created_at	datetime2	No	No

str_reports

Generated suspicious transaction report metadata and XML content/state.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	fraud_alert_id	bigint	No	No
3	str_no	varchar(50)	No	No
4	transaction_id	varchar(50)	No	No
5	account_id	varchar(50)	No	No
6	file_path	varchar(500)	Yes	No
7	generated_by	varchar(100)	Yes	No
8	generated_at	datetime2	No	No
9	status	varchar(30)	No	No

aml_shadow_predictions

Silent challenger/layered scores that cannot directly create production outcomes.

#	Column	SQL type	Nullable	Identity
1	shadow_prediction_id	uniqueidentifier	No	No
2	transaction_id	varchar(50)	No	No
3	account_id	varchar(50)	No	No
4	peer_group_code	varchar(50)	No	No
5	feature_version	varchar(30)	No	No
6	risk_policy_version	varchar(50)	No	No
7	legacy_risk_level	varchar(20)	No	No
8	legacy_suspicious	bit	No	No
9	legacy_anomaly_votes	int	No	No
10	layered_final_risk_score	float	No	No
11	layered_risk_level	varchar(20)	No	No
12	layered_suspicious	bit	No	No
13	hard_rule_override	bit	No	No
14	customer_behaviour_score	float	No	No
15	customer_behaviour_confidence	float	No	No
16	peer_behaviour_score	float	No	No
17	peer_behaviour_confidence	float	No	No
18	hst_score	float	Yes	No

aml_shadow_predictions (continued)

#	Column	SQL type	Nullable	Identity
19	hst_model_version	varchar(100)	Yes	No
20	online_ocsvm_score	float	Yes	No
21	online_ocsvm_model_version	varchar(100)	Yes	No
22	rule_score	float	No	No
23	suspicious_changed	bit	No	No
24	risk_level_changed	bit	No	No
25	alert_overlap	bit	No	No
26	component_scores_json	nvarchar(MAX)	No	No
27	triggered_rules_json	nvarchar(MAX)	No	No
28	reason_codes_json	nvarchar(MAX)	No	No
29	layered_result_json	nvarchar(MAX)	No	No
30	evaluated_at	datetime2	No	No
31	duration_ms	bigint	No	No

aml_shadow_scenario_labels

Expected labels for synthetic or controlled shadow scenarios.

#	Column	SQL type	Nullable	Identity
1	scenario_label_id	uniqueidentifier	No	No
2	transaction_id	varchar(50)	No	No
3	scenario_code	varchar(100)	No	No
4	expected_suspicious	bit	No	No
5	labeled_by	varchar(100)	No	No
6	created_at	datetime2	No	No

Training, registry, validation, and deployment**aml_training_runs**

Governed immutable training windows, export state, selected model, artifact, and failure metadata.

#	Column	SQL type	Nullable	Identity
1	training_run_id	uniqueidentifier	No	No
2	training_type	varchar(50)	No	No
3	feature_version	varchar(30)	No	No
4	model_type	varchar(50)	No	No
5	model_segment	varchar(50)	Yes	No
6	from_business_date	date	No	No
7	to_business_date	date	No	No
8	cutoff_timestamp	datetime2	No	No
9	requested_row_count	bigint	Yes	No
10	exported_row_count	bigint	Yes	No
11	learned_row_count	bigint	Yes	No
12	dataset_path	varchar(1000)	Yes	No
13	dataset_checksum	varchar(200)	Yes	No
14	base_model_version	varchar(100)	Yes	No
15	candidate_model_version	varchar(100)	Yes	No
16	status	varchar(30)	No	No
17	started_at	datetime2	Yes	No
18	completed_at	datetime2	Yes	No

aml_training_runs (continued)

#	Column	SQL type	Nullable	Identity
19	failure_reason	nvarchar(MAX)	Yes	No
20	created_at	datetime2	No	No
21	progress_stage	nvarchar(80)	Yes	No
22	progress_current	bigint	Yes	No
23	progress_total	bigint	Yes	No

training_runs

Compatibility training-run records used by earlier comparison/training flows.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	training_run_no	varchar(50)	No	No
3	dataset_partition_id	bigint	Yes	No
4	model_name	varchar(100)	Yes	No
5	training_status	varchar(30)	Yes	No
6	rows_used	int	Yes	No
7	feature_count	int	Yes	No
8	hyperparams_json	nvarchar(MAX)	Yes	No
9	training_started_at	datetime2	Yes	No
10	training_completed_at	datetime2	Yes	No
11	training_duration_ms	bigint	Yes	No
12	requested_by	varchar(100)	Yes	No
13	created_at	datetime2	No	No
14	artifacts_json	nvarchar(MAX)	Yes	No
15	response_status	varchar(30)	Yes	No
16	duration_ms	bigint	Yes	No
17	training_row_count	int	Yes	No
18	models_json	nvarchar(MAX)	Yes	No

training_runs (continued)

#	Column	SQL type	Nullable	Identity
19	message	nvarchar(MAX)	Yes	No
20	started_at	datetime2	Yes	No
21	completed_at	datetime2	Yes	No
22	run_no	varchar(50)	Yes	No
23	source	varchar(50)	Yes	No
24	status	varchar(30)	Yes	No

aml_model_registry

Immutable model versions, artifact paths/checksums, metrics, lifecycle state, and feature contract.

#	Column	SQL type	Nullable	Identity
1	model_version	varchar(100)	No	No
2	model_type	varchar(50)	No	No
3	model_segment	varchar(50)	Yes	No
4	feature_version	varchar(30)	No	No
5	training_run_id	uniqueidentifier	No	No
6	artifact_path	varchar(1000)	No	No
7	artifact_checksum	varchar(200)	No	No
8	status	varchar(30)	No	No
9	anomaly_rate	float	Yes	No
10	validation_row_count	bigint	Yes	No
11	alert_count	bigint	Yes	No
12	average_score	float	Yes	No
13	score_p95	float	Yes	No
14	score_p99	float	Yes	No
15	created_at	datetime2	No	No
16	approved_at	datetime2	Yes	No
17	deployed_at	datetime2	Yes	No
18	retired_at	datetime2	Yes	No

aml_model_registry (continued)

#	Column	SQL type	Nullable	Identity
19	dataset_checksum	varchar(200)	Yes	No
20	base_model_version	varchar(100)	Yes	No
21	feature_schema_checksum	varchar(200)	Yes	No
22	parameters_json	nvarchar(MAX)	Yes	No
23	metrics_json	nvarchar(MAX)	Yes	No
24	artifact_size_bytes	bigint	Yes	No
25	registered_by	varchar(100)	Yes	No

aml_active_models

Atomic active/previous model pointers by model type and segment.

#	Column	SQL type	Nullable	Identity
1	model_type	varchar(50)	No	No
2	model_segment	varchar(50)	Yes	No
3	model_segment_key	varchar(50)	No	No
4	active_model_version	varchar(100)	No	No
5	previous_model_version	varchar(100)	Yes	No
6	pointer_version	bigint	No	No
7	activated_by	varchar(100)	No	No
8	activated_at	datetime2	No	No

aml_model_validations

Candidate validation evidence and pass/fail governance result.

#	Column	SQL type	Nullable	Identity
1	validation_id	uniqueidentifier	No	No
2	model_version	varchar(100)	No	No
3	comparison_target	varchar(100)	No	No
4	window_started_at	datetime2	No	No
5	window_ended_at	datetime2	No	No
6	sample_count	bigint	No	No
7	candidate_anomaly_count	bigint	No	No
8	production_alert_count	bigint	No	No
9	overlap_count	bigint	No	No
10	candidate_only_count	bigint	No	No
11	production_only_count	bigint	No	No
12	candidate_anomaly_rate	float	No	No
13	production_alert_rate	float	No	No
14	agreement_rate	float	No	No
15	alert_jaccard	float	Yes	No
16	average_score	float	Yes	No
17	score_stddev	float	Yes	No
18	score_p50	float	Yes	No

aml_model_validations (continued)

#	Column	SQL type	Nullable	Identity
19	score_p95	float	Yes	No
20	score_p99	float	Yes	No
21	daily_anomaly_rate_stddev	float	Yes	No
22	reviewed_overlap_count	bigint	No	No
23	false_positive_overlap_count	bigint	No	No
24	str_overlap_count	bigint	No	No
25	reviewed_precision	float	Yes	No
26	validation_status	varchar(30)	No	No
27	failure_reason	varchar(1000)	Yes	No
28	metrics_json	nvarchar(MAX)	No	No
29	validated_by	varchar(100)	No	No
30	validated_at	datetime2	No	No

aml_model_deployments

Promotion and rollback event history for model versions.

#	Column	SQL type	Nullable	Identity
1	deployment_id	uniqueidentifier	No	No
2	action_id	uniqueidentifier	No	No
3	deployment_action	varchar(20)	No	No
4	model_type	varchar(50)	No	No
5	model_segment	varchar(50)	Yes	No
6	previous_model_version	varchar(100)	Yes	No
7	activated_model_version	varchar(100)	No	No
8	reason	nvarchar(1000)	No	No
9	performed_by	varchar(100)	No	No
10	performed_at	datetime2	No	No

aml_layered_validations

Validation reports for the complete layered risk architecture.

#	Column	SQL type	Nullable	Identity
1	validation_id	uniqueidentifier	No	No
2	risk_policy_version	varchar(50)	No	No
3	peer_group_code	varchar(50)	Yes	No
4	window_started_at	datetime2	No	No
5	window_ended_at	datetime2	No	No
6	sample_count	bigint	No	No
7	observation_days	int	No	No
8	legacy_alert_count	bigint	No	No
9	layered_alert_count	bigint	No	No
10	overlap_count	bigint	No	No
11	alert_jaccard	float	Yes	No
12	top_risk_overlap_rate	float	Yes	No
13	daily_alert_rate_stddev	float	Yes	No
14	max_segment_daily_stddev	float	Yes	No
15	synthetic_scenario_recall	float	Yes	No
16	reviewed_false_positive_rate	float	Yes	No
17	prediction_latency_p95_ms	float	Yes	No
18	average_incremental_update_ms	float	Yes	No

aml_layered_validations (continued)

#	Column	SQL type	Nullable	Identity
19	hst_availability_rate	float	No	No
20	online_ocsvm_availability_rate	float	No	No
21	validation_status	varchar(30)	No	No
22	blocking_reasons_json	nvarchar(MAX)	No	No
23	warnings_json	nvarchar(MAX)	No	No
24	metrics_json	nvarchar(MAX)	No	No
25	validated_by	varchar(100)	No	No
26	validated_at	datetime2	No	No

aml_layered_deployment_pointers

Active layered architecture pointer, locked policy/models, and rollout percentage.

#	Column	SQL type	Nullable	Identity
1	peer_group_code	varchar(50)	No	No
2	deployment_mode	varchar(40)	No	No
3	risk_policy_version	varchar(50)	No	No
4	hst_model_version	varchar(100)	No	No
5	online_ocsvm_model_version	varchar(100)	No	No
6	validation_id	uniqueidentifier	No	No
7	canary_percentage	int	No	No
8	pointer_version	bigint	No	No
9	activated_by	varchar(100)	No	No
10	activated_at	datetime2	No	No

aml_layered_deployment_events

Audit history for layered promotion, expansion, and rollback.

#	Column	SQL type	Nullable	Identity
1	deployment_id	uniqueidentifier	No	No
2	action_id	uniqueidentifier	No	No
3	deployment_action	varchar(30)	No	No
4	peer_group_code	varchar(50)	No	No
5	previous_mode	varchar(40)	Yes	No
6	activated_mode	varchar(40)	No	No
7	risk_policy_version	varchar(50)	No	No
8	hst_model_version	varchar(100)	No	No
9	online_ocsvm_model_version	varchar(100)	No	No
10	validation_id	uniqueidentifier	No	No
11	previous_canary_percentage	int	Yes	No
12	activated_canary_percentage	int	No	No
13	reason	varchar(1000)	No	No
14	performed_by	varchar(100)	No	No
15	performed_at	datetime2	No	No

model_versions

Compatibility registry for earlier partition-model lifecycle workflows.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	model_version_no	varchar(50)	No	No
3	training_run_id	bigint	No	No
4	model_name	varchar(100)	No	No
5	partition_size	int	No	No
6	artifact_base_path	varchar(500)	No	No
7	feature_columns_path	varchar(500)	Yes	No
8	scaler_path	varchar(500)	Yes	No
9	model_path	varchar(500)	No	No
10	metrics_json	nvarchar(MAX)	Yes	No
11	is_active	bit	No	No
12	created_at	datetime2	No	No
13	dataset_partition_id	bigint	Yes	No
14	lifecycle_status	varchar(30)	No	No
15	promoted_at	datetime2	Yes	No
16	promoted_by	varchar(100)	Yes	No

Research and comparison

aml_growth_studies

Unsupervised data-growth study status and serialized result.

#	Column	SQL type	Nullable	Identity
1	study_id	uniqueidentifier	No	No
2	training_run_id	uniqueidentifier	No	No
3	status	varchar(30)	No	No
4	feature_version	varchar(30)	Yes	No
5	dataset_rows	bigint	Yes	No
6	feature_count	int	Yes	No
7	partition_percentages	varchar(100)	Yes	No
8	methodology_json	nvarchar(MAX)	Yes	No
9	requested_by	varchar(100)	Yes	No
10	failure_reason	nvarchar(8000)	Yes	No
11	started_at	datetime2	Yes	No
12	completed_at	datetime2	Yes	No
13	created_at	datetime2	No	No

aml_growth_metrics

Per-model/per-partition unsupervised quality, stability, rate, latency, and capacity metrics.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	study_id	uniqueidentifier	No	No
3	detector	varchar(50)	No	No
4	partition_percentage	int	No	No
5	partition_rows	bigint	Yes	No
6	training_rows	bigint	Yes	No
7	learned_rows	bigint	Yes	No
8	evaluation_rows	bigint	Yes	No
9	excess_mass_auc	float	Yes	No
10	score_skewness	float	Yes	No
11	rank_stability	float	Yes	No
12	anomaly_rate	float	Yes	No
13	alert_count	bigint	Yes	No
14	threshold	float	Yes	No
15	average_score	float	Yes	No
16	score_p50	float	Yes	No
17	score_p95	float	Yes	No
18	score_p99	float	Yes	No

aml_growth_metrics (continued)

#	Column	SQL type	Nullable	Identity
19	training_duration_ms	float	Yes	No
20	rows_per_second	float	Yes	No
21	bounded_training_sample	bit	No	No
22	created_at	datetime2	No	No

aml_supervised_growth_studies

Supervised chronological comparison status and cached result.

#	Column	SQL type	Nullable	Identity
1	study_id	uniqueidentifier	No	No
2	training_run_id	uniqueidentifier	No	No
3	status	varchar(30)	No	No
4	result_json	nvarchar(MAX)	Yes	No
5	requested_by	varchar(100)	Yes	No
6	failure_reason	nvarchar(8000)	Yes	No
7	started_at	datetime2	Yes	No
8	completed_at	datetime2	Yes	No
9	created_at	datetime2	No	No

aml_agreement_studies

Unsupervised model agreement and overlap study cache.

#	Column	SQL type	Nullable	Identity
1	study_id	uniqueidentifier	No	No
2	training_run_id	uniqueidentifier	No	No
3	status	varchar(30)	No	No
4	evaluated_rows	bigint	Yes	No
5	model_count	int	Yes	No
6	result_json	nvarchar(MAX)	Yes	No
7	requested_by	varchar(100)	Yes	No
8	failure_reason	nvarchar(8000)	Yes	No
9	started_at	datetime2	Yes	No
10	completed_at	datetime2	Yes	No
11	created_at	datetime2	No	No

dataset_partitions

Chronological percentage partitions and row counts for model-growth comparison.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	uploaded_dataset_id	bigint	No	No
3	partition_no	varchar(50)	No	No
4	partition_label	varchar(100)	No	No
5	partition_size	int	No	No
6	ordering_strategy	varchar(50)	No	No
7	start_row_no	int	No	No
8	end_row_no	int	No	No
9	created_at	datetime2	No	No

comparison_runs

Earlier scenario-comparison run header retained in the backend.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	comparison_run_no	varchar(50)	No	No
3	uploaded_dataset_id	bigint	No	No
4	scenario_set_id	bigint	No	No
5	selected_partition_sizes	varchar(200)	No	No
6	selected_models	varchar(500)	No	No
7	run_status	varchar(30)	No	No
8	requested_by	varchar(100)	Yes	No
9	started_at	datetime2	Yes	No
10	completed_at	datetime2	Yes	No
11	created_at	datetime2	No	No

comparison_results

Earlier model-comparison metrics retained for compatibility.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	comparison_run_id	bigint	No	No
3	scenario_id	bigint	No	No
4	dataset_partition_id	bigint	No	No
5	model_version_id	bigint	No	No
6	model_name	varchar(100)	No	No
7	raw_prediction	int	Yes	No
8	anomaly_vote	int	No	No
9	risk_level	varchar(20)	Yes	No
10	suspicious	bit	Yes	No
11	recommended_action	varchar(50)	Yes	No
12	score_value	float	Yes	No
13	decision_value	float	Yes	No
14	reasons_json	nvarchar(MAX)	Yes	No
15	response_json	nvarchar(MAX)	Yes	No
16	prediction_duration_ms	bigint	Yes	No
17	created_at	datetime2	No	No

comparison_scenarios

Earlier reusable comparison scenario definitions.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	scenario_no	varchar(50)	No	No
3	scenario_set_id	bigint	No	No
4	scenario_name	varchar(150)	No	No
5	scenario_type	varchar(100)	Yes	No
6	transaction_json	nvarchar(MAX)	No	No
7	customer_json	nvarchar(MAX)	Yes	No
8	account_profile_json	nvarchar(MAX)	No	No
9	expected_notes	varchar(500)	Yes	No
10	created_at	datetime2	No	No

scenario_sets

Earlier scenario-set metadata retained although its UI page is inactive.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	scenario_set_no	varchar(50)	No	No
3	name	varchar(150)	No	No
4	description	varchar(500)	Yes	No
5	created_by	varchar(100)	Yes	No
6	created_at	datetime2	No	No

anomaly_configs

Compatibility anomaly-policy records retained for earlier configuration flows.

#	Column	SQL type	Nullable	Identity
1	id	bigint	No	Yes
2	config_no	varchar(50)	No	No
3	config_name	varchar(150)	No	No
4	enabled_models_json	nvarchar(MAX)	No	No
5	voting_strategy	varchar(50)	No	No
6	suspicious_vote_threshold	int	No	No
7	high_risk_vote_threshold	int	No	No
8	medium_risk_vote_threshold	int	No	No
9	gating_enabled	bit	No	No
10	gating_config_json	nvarchar(MAX)	Yes	No
11	is_active	bit	No	No
12	created_by	varchar(100)	Yes	No
13	created_at	datetime2	No	No
14	updated_at	datetime2	Yes	No
15	dataset_partition_id	bigint	Yes	No
16	artifact_base_path	varchar(500)	Yes	No

Appendix C. Runtime Configuration Catalog

The app_config table is the runtime configuration authority. Values are intentionally omitted here because they are mutable; the key, type, and purpose are the stable defense reference.

AML training, export, registry, and validation

Key	Type	Purpose
aml.autoencoder.enabled	BOOLEAN	Generate new Autoencoder candidates during training runs.
aml.cluster_outlier.enabled	BOOLEAN	Generate Behavioral Cluster Outlier candidates during unsupervised training.
aml.elliptic_envelope.contamination	DECIMAL	Expected proportion of anomalies in training data - sets the decision boundary.
aml.elliptic_envelope.enabled	BOOLEAN	Generate new Elliptic Envelope candidates during training runs.
aml.elliptic_envelope.max_training_rows	INTEGER	Maximum rows loaded from historical data for each training run.
aml.elliptic_envelope.parquet_batch_size	INTEGER	Feature rows decoded per Python batch during training.
aml.export.base_path	STRING	Base directory for versioned Parquet training datasets.
aml.export.chunk_size	INTEGER	Maximum eligible feature rows read from SQL Server per keyset-paginated chunk.
aml.export.rows_per_file	INTEGER	Maximum rows written to each Parquet part file.
aml.isolation_forest.enabled	BOOLEAN	Generate new Isolation Forest candidates during training runs.
aml.isolation_forest.parquet_batch_size	INTEGER	Feature rows decoded per Python batch during training.
aml.isolation_forest.threshold_quantile	DECIMAL	Training score quantile used as the anomaly decision boundary.
aml.layered_validation.max_validation_age_days	INTEGER	Maximum age of a passing layered validation used for promotion.
aml.layered_validation.max_alert_rate	DECIMAL	Maximum acceptable layered suspicious rate.
aml.layered_validation.max_alert_volume_increase	DECIMAL	Maximum relative alert-volume increase over legacy.
aml.layered_validation.max_average_incremental_update_ms	DECIMAL	Maximum average completed incremental update duration.

AML training, export, registry, and validation (continued)

Key	Type	Purpose
aml.layered_validation.max_daily_rate_stddev	DECIMAL	Maximum daily layered-alert-rate standard deviation.
aml.layered_validation.max_p95_latency_ms	DECIMAL	Maximum layered shadow p95 latency in milliseconds.
aml.layered_validation.max_reviewed_false_positive_rate	DECIMAL	Maximum false-positive share among reviewed layered alerts.
aml.layered_validation.max_segment_daily_stddev	DECIMAL	Maximum daily alert-rate deviation in any segment.
aml.layered_validation.min_legacy_alerts	INTEGER	Minimum legacy alerts needed for overlap comparison.
aml.layered_validation.min_model_availability	DECIMAL	Minimum score availability for each production ML model.
aml.layered_validation.min_observation_days	INTEGER	Minimum distinct observation days required.
aml.layered_validation.min_reviewed_alerts	INTEGER	Minimum reviewed layered alerts required.
aml.layered_validation.min_rows	INTEGER	Minimum shadow comparisons required.
aml.layered_validation.min_synthetic_recall	DECIMAL	Minimum recall on labeled synthetic AML scenarios.
aml.layered_validation.min_synthetic_scenarios	INTEGER	Minimum expected-positive synthetic scenarios required.
aml.layered_validation.min_top_risk_overlap	DECIMAL	Minimum legacy-alert coverage among the layered top one percent.
aml.lof.max_training_rows	INTEGER	Maximum rows loaded from historical data for each training run.
aml.lof.n_neighbors	INTEGER	Local neighbourhood size - smaller values are more sensitive to local patterns.
aml.lof.parquet_batch_size	INTEGER	Feature rows decoded per Python batch during training.
aml.lof.threshold_quantile	DECIMAL	Training score quantile used as the anomaly decision boundary.

AML training, export, registry, and validation (continued)

Key	Type	Purpose
aml.model.artifact_base_path	STRING	Base directory containing immutable candidate model bundles.
aml.ocsvm_batch.enabled	BOOLEAN	Generate new batch One-Class SVM candidates during training runs.
aml.ocsvm_batch.gamma	DECIMAL	Kernel coefficient - lower values produce smoother boundaries.
aml.ocsvm_batch.max_training_rows	INTEGER	Maximum rows used for fitting - SVM training scales quadratically so keep this conservative.
aml.ocsvm_batch.nu	DECIMAL	Upper bound on the fraction of training errors and support vectors.
aml.ocsvm_batch.parquet_batch_size	INTEGER	Feature rows decoded per Python batch during training.
aml.ocsvm_batch.threshold_quantile	DECIMAL	Training score quantile used as the anomaly decision boundary.
aml.pca.batch_size	INTEGER	Rows per incremental PCA fitting step - must be = max components.
aml.pca.enabled	BOOLEAN	Generate new PCA Reconstruction candidates during training runs.
aml.pca.max_training_rows	INTEGER	Maximum rows loaded from historical data for each training run.
aml.pca.parquet_batch_size	INTEGER	Feature rows decoded per Python batch during training.
aml.pca.threshold_quantile	DECIMAL	Reconstruction error quantile used as the anomaly decision boundary.
aml.terminal_risk.delay_days	INTEGER	Investigation delay applied only to confirmed-label terminal risk features.
aml.terminal_risk.enabled	BOOLEAN	Enable point-in-time terminal history features.
aml.terminal_risk.minimum_transactions	INTEGER	Minimum 30-day terminal volume required for terminal risk availability.
aml.terminal_risk.smoothing_strength	DECIMAL	Bayesian prior strength for terminal confirmed-fraud rates.

AML training, export, registry, and validation (continued)

Key	Type	Purpose
aml.training.schedule.batch	STRING	Retraining cadence for batch production detectors.
aml.training.schedule.incremental	STRING	Retraining cadence for incremental production detectors.
aml.validation.max_anomaly_rate	DECIMAL	Maximum acceptable challenger anomaly rate.
aml.validation.max_daily_rate_stddev	DECIMAL	Maximum acceptable daily anomaly-rate standard deviation.
aml.validation.min_anomaly_rate	DECIMAL	Minimum acceptable challenger anomaly rate.
aml.validation.min_reviewed_alerts	INTEGER	Minimum reviewed overlap alerts before analyst precision becomes a gate.
aml.validation.min_reviewed_precision	DECIMAL	Minimum STR share among reviewed overlapping alerts.
aml.validation.min_rows	INTEGER	Minimum silent predictions required to validate a candidate.

ML and cold-start compatibility

Key	Type	Purpose
ml.cluster_outlier.batch_size	INTEGER	Rows processed per MiniBatch K-Means update.
ml.cluster_outlier.contamination	DECIMAL	Expected cluster-conditional anomaly fraction.
ml.cluster_outlier.n_clusters	INTEGER	Number of broad transaction-behaviour groups.
ml.cold_start.default_recommended_action	STRING	Recommended action returned during cold start
ml.cold_start.default_risk_level	STRING	Risk level returned during cold start
ml.cold_start.enabled	STRING	Enable or disable cold start handling
ml.cold_start.high_login_attempt_threshold	STRING	Optional login-attempt threshold for cold start warning
ml.cold_start.max_amount_multiplier_for_warn	STRING	Optional multiplier for warning on unusually high amount during cold start
ml.cold_start.reason_message	STRING	Reason shown when cold start is triggered
ml.cold_start.use_rules_before_ml	STRING	Whether simple rules should still run during cold start
ml.extra_trees.enabled	BOOLEAN	Generate Extra Trees candidates during supervised training.
ml.extra_trees.max_features	DECIMAL	Feature fraction considered at each Extra Trees split.
ml.extra_trees.min_samples_leaf	INTEGER	Minimum labelled rows in each Extra Trees terminal leaf.
ml.extra_trees.n_estimators	INTEGER	Number of highly randomized Extra Trees estimators.
ml.iso.contamination	STRING	IsolationForest contamination (auto or float like 0.01)
ml.iso.n_estimators	INT	IsolationForest n_estimators

ML and cold-start compatibility (continued)

Key	Type	Purpose
ml.min_transaction_count_before_predict	INT	Minimum prior transaction count required before ML prediction
ml.random_forest.enabled	BOOLEAN	Generate Random Forest candidates during supervised training.
ml.random_forest.max_depth	INTEGER	Maximum Random Forest tree depth.
ml.random_forest.min_samples_leaf	INTEGER	Minimum rows in a Random Forest terminal leaf.
ml.random_forest.n_estimators	INTEGER	Number of Random Forest trees.
ml.random_state	INT	Random seed used for training where applicable
ml.stacked_ensemble.enabled	BOOLEAN	Train the leakage-safe temporal supervised stack.
ml.svm.gamma	STRING	OneClassSVM gamma (scale/auto/float)
ml.svm.kernel	STRING	OneClassSVM kernel
ml.svm.nu	DECIMAL	OneClassSVM nu
ml.xgboost.colsample_bytree	DECIMAL	XGBoost feature sample fraction.
ml.xgboost.enabled	BOOLEAN	Generate XGBoost candidates during supervised training.
ml.xgboost.learning_rate	DECIMAL	XGBoost learning rate.
ml.xgboost.max_depth	INTEGER	Maximum XGBoost tree depth.
ml.xgboost.min_child_weight	DECIMAL	Minimum XGBoost child weight used to reduce minority-class overfitting.
ml.xgboost.n_estimators	INTEGER	Number of XGBoost boosting trees.

ML and cold-start compatibility (continued)

Key	Type	Purpose
ml.xgboost.reg_alpha	DECIMAL	XGBoost L1 regularization for weak or noisy feature effects.
ml.xgboost.reg_lambda	DECIMAL	XGBoost L2 regularization for stable imbalanced classification.
ml.xgboost.subsample	DECIMAL	XGBoost row sample fraction.

Research comparison

Key	Type	Purpose
aml.research.autoencoder_hidden_layer_sizes	TEXT	Comma-separated neuron counts per layer. The middle value is the bottleneck - smaller forces tighter compression and stronger anomaly signal.
aml.research.autoencoder_max_iter	INTEGER	Maximum gradient descent epochs - early stopping may terminate training before this limit.
aml.research.autoencoder_max_training_rows	INTEGER	Maximum rows used for fitting the autoencoder.
aml.research.cluster_outlier_max_training_rows	INTEGER	Maximum rows used by Behavioral Cluster Outlier in each growth partition.
aml.research.holdout_fraction	DECIMAL	Newest fraction of each partition reserved for evaluation; the remainder trains each model.
aml.research.incremental_pca_max_components	INTEGER	Principal components retained - higher values preserve more variance but slow scoring.
aml.research.isolation_forest_max_samples	INTEGER	Rows sampled per tree - default 256 is too small for large datasets; 10,000 gives much stronger isolation signal.
aml.research.isolation_forest_max_training_rows	INTEGER	Maximum rows loaded from historical data for each training run.
aml.research.isolation_forest_n_estimators	INTEGER	More trees improve anomaly score stability at higher training time cost.
aml.research.maximum_evaluation_rows	INTEGER	Maximum holdout rows scored per model per partition during comparison.
aml.research.minimum_rows	INTEGER	Skip data partitions with fewer rows than this limit during comparison training.
aml.research.ocsvm_max_training_rows	INTEGER	Maximum rows used for fitting - set high to train on all available data.
aml.research.random_seed	INTEGER	Shared reproducibility seed for all model training and comparison runs.

Risk policy and rules

Key	Type	Purpose
aml.risk.layered_shadow_enabled	BOOLEAN	Allow layered scoring to run without changing production outcomes.
aml.risk.legacy_comparison_enabled	BOOLEAN	Retain legacy production scoring for shadow comparison.
aml.risk.ml_model_allocations_json	JSON	Selected production ML models and their relative ensemble weights.
aml.risk.policy.version	STRING	Active layered risk policy version.
aml.risk.rules.extreme_customer_amount_ratio	DECIMAL	Multiplier vs 30-day customer average that triggers the extreme-amount rule.
aml.risk.rules.high_balance_ratio	DECIMAL	Transaction-to-balance ratio threshold that triggers the high-balance rule.
aml.risk.rules.high_customer_amount_ratio	DECIMAL	Multiplier vs 30-day customer average that triggers the high-amount rule.
aml.risk.rules.high_expected_turnover_ratio	DECIMAL	Transaction-to-expected-turnover ratio threshold that triggers the turnover rule.
aml.risk.rules.high_tx_count_1h	INTEGER	Transaction count in 1 hour that triggers high velocity rule.
aml.risk.rules.multi_beneficiary_count_1h	INTEGER	Unique beneficiary count in 1 hour that triggers the multiple-beneficiary rule.
aml.risk.rules.rapid_tx_count_10m	INTEGER	Transaction count in 10 minutes that triggers rapid velocity rule.
aml.risk.rules.repeated_amount_count_24h	INTEGER	Identical-amount transaction count in 24 h that triggers the repeated-amount rule.
aml.risk.rules.reporting_threshold	DECIMAL	Regulatory reporting threshold used in structuring detection.
aml.risk.rules structuring_count_24h	INTEGER	Minimum below-threshold transaction count in 24 h to flag structuring.
aml.risk.threshold.high	DECIMAL	Inclusive HIGH risk threshold.
aml.risk.threshold.low	DECIMAL	Inclusive LOW risk threshold.

Risk policy and rules (continued)

Key	Type	Purpose
aml.risk.threshold.medium	DECIMAL	Inclusive MEDIUM risk and automatic case threshold.
aml.risk.weight.customer_behaviour	DECIMAL	Customer behaviour contribution to final risk.
aml.risk.weight.customer_behaviour.amount	DECIMAL	Amount deviation sub-weight within customer behaviour scoring.
aml.risk.weight.customer_behaviour.frequency	DECIMAL	Velocity burst sub-weight within customer behaviour scoring.
aml.risk.weight.customer_behaviour.novelty	DECIMAL	Novelty (new beneficiary/location/channel/device) sub-weight within customer behaviour scoring.
aml.risk.weight.customer_behaviour.time_gap	DECIMAL	Rapid time-gap sub-weight within customer behaviour scoring.
aml.risk.weight.customer_behaviour.unusual_hour	DECIMAL	Unusual transaction hour sub-weight within customer behaviour scoring.
aml.risk.weight.half_space_trees	DECIMAL	Half-Space Trees normalized-score weight.
aml.risk.weight.ml_ensemble	DECIMAL	ML model ensemble contribution to final risk.
aml.risk.weight.online_ocsvm	DECIMAL	Online One-Class SVM normalized-score weight.
aml.risk.weight.peer_behaviour	DECIMAL	Peer behaviour contribution to final risk.
aml.risk.weight.peer_behaviour.amount	DECIMAL	Amount deviation sub-weight within peer behaviour scoring.
aml.risk.weight.peer_behaviour.expected_turnover	DECIMAL	Expected turnover sub-weight within peer behaviour scoring.
aml.risk.weight.peer_behaviour.frequency	DECIMAL	Frequency percentile sub-weight within peer behaviour scoring.
aml.risk.weight.rules	DECIMAL	Deterministic AML rules contribution to final risk.

Supervised learning

Key	Type	Purpose
ml.supervised.stacking_protocol	TEXT	Fit stack weights on the first calibration half and its threshold on the second half.
ml.supervised.threshold_beta	DECIMAL	F-beta used to calibrate each supervised model decision threshold.

System mode

Key	Type	Purpose
system.learning_mode	SELECT	Global learning mode. Allowed values: UNSUPERVISED or SUPERVISED.

Appendix D. Code Navigation Cheat Sheet

Question	Start here
Where is login/register logic?	auth/web/AuthController.java; auth/service/AuthService.java
Where is JWT verified?	auth/security/JwtAuthenticationFilter.java; auth/service/JwtService.java
Where are roles enforced?	auth/security/SecurityConfig.java and Angular core/auth.guard.ts
Where is file upload parsed?	uploads/service/ExcelBulkUploadService.java
Where is a transaction scored?	transactions/service/TransactionCreateService.java; aml/prediction/AmlPredictionOrchestrator.java
Where are features calculated?	aml/feature/application/FeatureEngineeringService.java
Where are features persisted?	aml/feature/infrastructure/FeaturePersistenceService.java
Where is customer behavior scored?	aml/behaviour/customer/CustomerBehaviourScorer.java
Where is peer behavior scored?	aml/behaviour/peer/PeerBehaviourScorer.java
Where are AML rules evaluated?	aml/rules/engine/DeterministicAmlRuleEngine.java
Where is final score calculated?	aml/risk/application/WeightedRiskAggregationEngine.java
Where is risk policy loaded?	aml/risk/infrastructure/AppConfigRiskPolicyRepository.java
Where are labels changed?	transactions/service/TransactionFraudLabelService.java
Where are cases and STR handled?	cases/service/CaseManagementService.java; cases/service/StrXmlService.java
Where is the training pipeline?	aml/training/application/TrainingPipelineJobLauncher.java
Where is Parquet exported?	aml/training/application/TrainingDatasetExportService.java
Where are candidate models mapped?	aml/training/application/ProductionCandidateTrainingService.java
Where is model lifecycle handled?	aml/training/application/AmlModelRegistryService.java; aml/deployment/application/ModelDeploymentService.java
Where is unsupervised comparison?	aml/research/application/GrowthAnalysisService.java; fraud-ml- service/app/research/growth_analysis.py
Where is supervised comparison?	aml/research/application/SupervisedGrowthStudyService.java; fraud-ml- service/app/research/supervised_growth_analysis.py
Where is full policy backtested?	fraud-ml-service/app/research/risk_policy_backtest.py
Where are models trained in Python?	fraud-ml-service/app/training.py; app/supervised_training.py
Where are Python endpoints?	fraud-ml-service/app/main.py
Where are Angular routes?	fraud-transaction-ui/src/app/app.routes.ts
Where are Angular API calls/types?	fraud-transaction-ui/src/app/core

Final defense rule: If you are unsure, state what the current code proves, distinguish prototype behavior from production assurance, and avoid promising accuracy or scale that has not been measured on real bank data.